Informe Final - TPO POO Etapa 2

Integrantes:

- Dolores Cerrudo Amoedo
- Germana Mendoza

1. Distribución de Tareas

El desarrollo de la Etapa 2 se dividió de manera equitativa utilizando repositorios y ramas en GitHub para aislar los desarrollos antes de su integración:

- **Dolores Cerrudo Amoedo:** Responsable del modelado e implementación de las entidades Pasajero y Cliente, la jerarquía de la clase abstracta Empleado, toda la lógica core de la clase Reserva, la CalculadoraTarifa y el ControladorReserva para orquestar los flujos. También tuvo a cargo la lógica de registro de pagos y recargos.
- **Germana Mendoza:** Encargada del modelado e implementación del catálogo de Tour (con su jerarquía de herencia), la gestión de la entidad Salida, el diseño polimórfico de las políticas de cancelación, la creación del paquete de excepciones personalizadas, la persistencia binaria de los datos (GestorPersistencia) y el diseño de las pruebas unitarias.

2. Cambios respecto del diseño original

Durante la transición del diseño inicial a la codificación final en Java, se realizaron refactorizaciones clave para garantizar el cumplimiento de la arquitectura Modelo-Vista-Controlador (MVC) y la fidelidad absoluta al diagrama de clases definitivo:

- **Inclusión de Controladores (Patrón Controlador - GRASP):** Para evitar que la clase central Agencia se transformara en un "Objeto Dios" con un acoplamiento excesivo, se crearon las clases ControladorReserva, ControladorOperaciones y ControladorAdministracion. Estos orquestan los flujos de información entre la vista y el dominio.
- **Delegación de cálculos (Responsabilidad Única SRP - SOLID):** La lógica matemática de precios y descuentos se extrajo de Reserva hacia la clase delegada CalculadoraTarifa, asegurando que Reserva solo tenga una razón para cambiar.
- **Aplicación de Polimorfismo (GRASP):** Se reemplazaron las estructuras condicionales para las devoluciones mediante la interfaz PoliticaCancelacion y sus implementaciones concretas (CancelacionFlexible, CancelacionEstricta), permitiendo un código abierto a la extensión y cerrado a la modificación.
- **Limpieza del Modelo de Dominio:** Se eliminaron clases e interfaces "fantasma" que no figuraban en el diagrama estático final (como LiquidacionGuia y Promocion) y se removieron marcas de persistencia (Serializable) de las clases de negocio puro, delegando el guardado exclusivamente al GestorPersistencia.

3. Dificultades Encontradas

- **Control de Versiones y Trabajo Colaborativo:** La sincronización de cambios en GitHub presentó desafíos iniciales, particularmente al integrar ramas con

dependencias cruzadas (como la asociación entre Reserva y Salida). Requirió establecer una comunicación activa previa a la creación y aprobación de cada *Pull Request*.

- **Alineación Estricta Código-UML:** Fue un desafío purgar el código de atributos auxiliares, constantes, *getters/setters* innecesarios y dependencias externas que resultaban cómodas para la programación pero que rompían la fidelidad académica con el diagrama de clases validado en la Etapa 1.
- **Gestión del Tiempo y Fechas:** El cálculo exacto de edades, vigencias de tarifas y anticipación de reservas exigió comprender y manipular correctamente la API nativa java.time (utilizando LocalDate y ChronoUnit) para evitar inconsistencias por años bisiestos o duraciones irregulares de meses.

4. Reflexión del Grupo

El desarrollo de este TPO evidenció de forma práctica la importancia crítica de definir un modelo de dominio sólido antes de escribir la primera línea de código. La aplicación consciente de los patrones GRASP y los principios SOLID desde la fase de diseño facilitó enormemente la distribución de responsabilidades y la modularización. Mantener la capa de presentación aislada de las reglas de negocio, a través del patrón MVC, nos garantizó un sistema limpio y escalable. Además, el uso obligatorio de Git y GitHub nos permitió vivenciar un entorno de desarrollo profesional, destacando el valor de la documentación y el trabajo en equipo organizado.